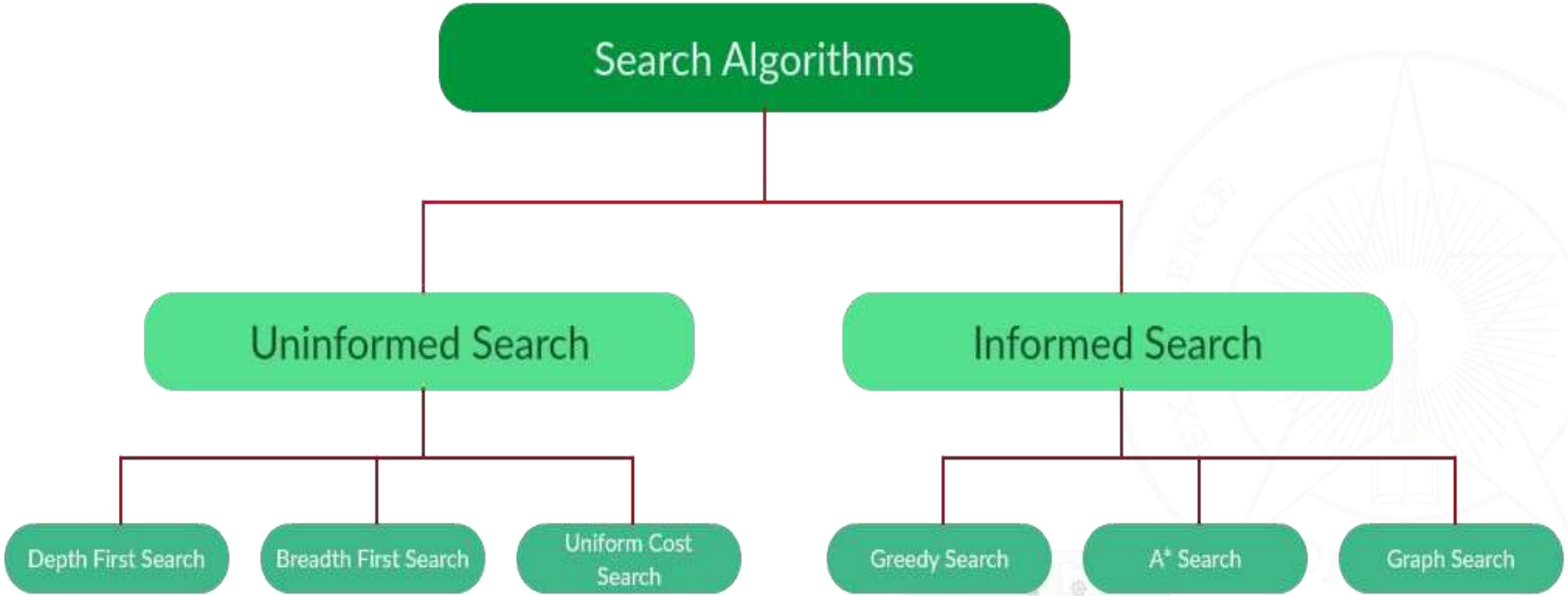
UNIT 02 - Part 2

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Informed (Heuristic) Search Strategies

Heuristic

Heuristic is an estimated knowledge or rule of thumb that helps the search algorithm find the solution quickly.

Or it means a **smart guess** or **shortcut knowledge** used to reach the solution faster.
In **informed search**, the computer does not search blindly.
It uses a **heuristic value** to decide **which path looks closer to the goal**.
Example:
Suppose you want to go from **College to Home**.
You have many roads:
Road A: 10 km
Road B: 5 km
Road C: 20 km
A heuristic will suggest:
Choose Road B because it seems closer to the destination.
So, heuristic helps to select the **most promising path**.

Heuristics function:

Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by h(n), and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Admissibility of the heuristic function is given as:

$h(n) <= h^*(n)$

Here h(n) is heuristic cost, and h*(n) is the actual cost. Hence heuristic cost should be less than or equal to the actual cost.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues unit a goal state is found.

In the informed search we will discuss two main algorithms which are given below:

- **Best First Search Algorithm(Greedy search)**
- **A* Search Algorithm**

Best-first Search Algorithm (Greedy Search):

Greedy best-first search algorithm always selects the path which appears best at that moment.

It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms.

With the help of best-first search, at each step, we can choose the most promising node.

In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$f(n) = g(n)$.

Were, $h(n)$= estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

Step 1: Place the starting node into the OPEN list.
Step 2: If the OPEN list is empty, Stop and return failure.
Step 3: Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
Step 4: Expand the node n, and generate the successors of node n.
Step 5: Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
Step 6: For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
Step 7: Return to Step 2.

Advantages:

Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.

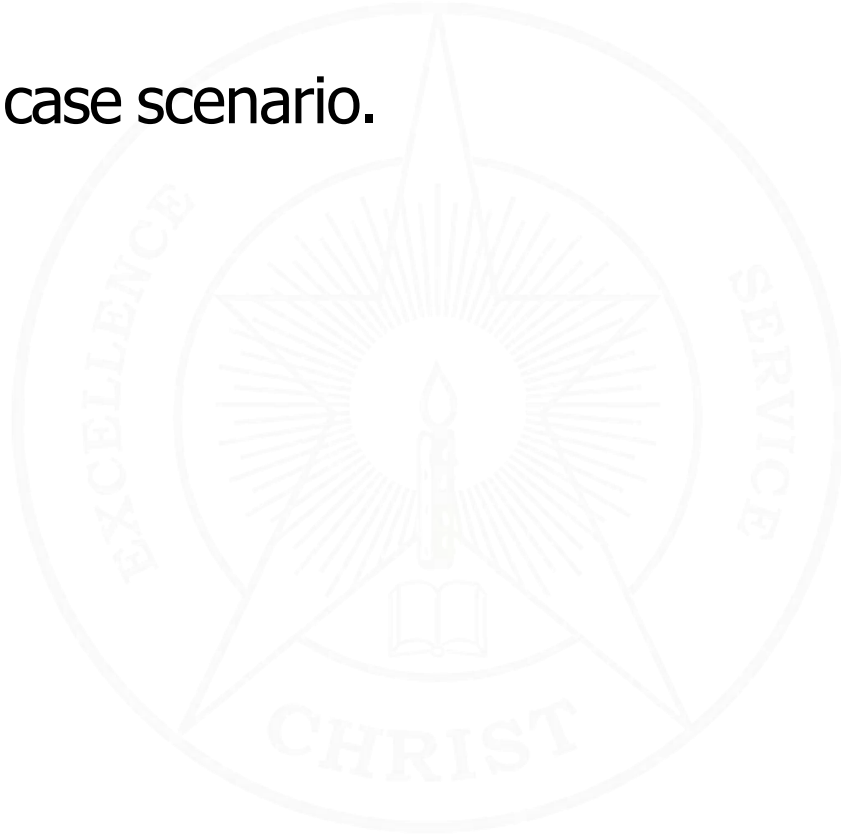
This algorithm is more efficient than BFS and DFS algorithms.

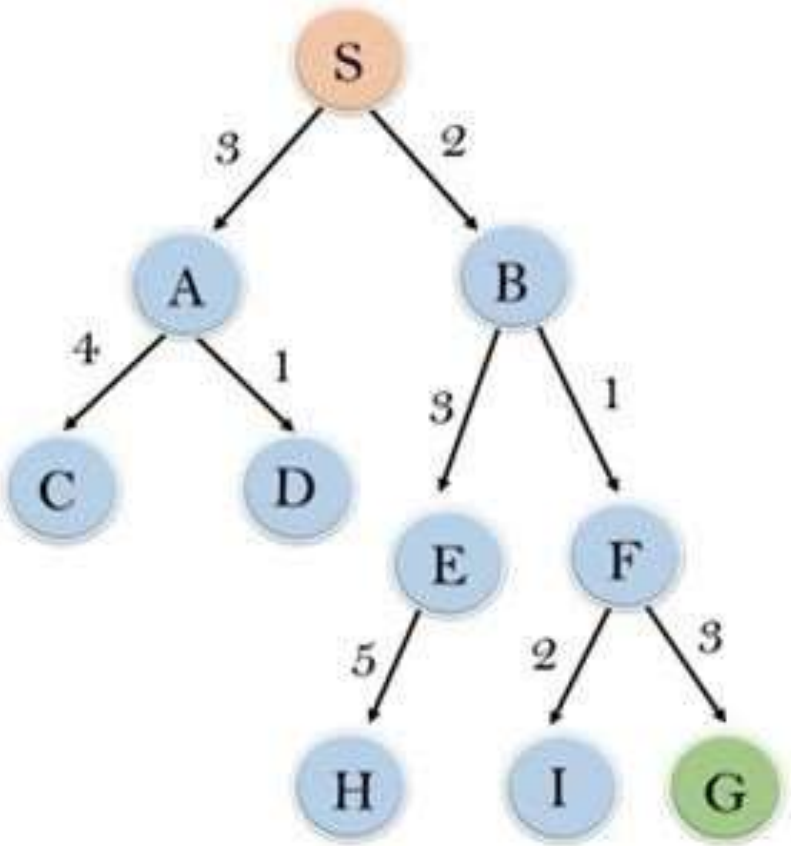
Disadvantages:

It can behave as an unguided depth-first search in the worst case scenario.

It can get stuck in a loop as DFS.

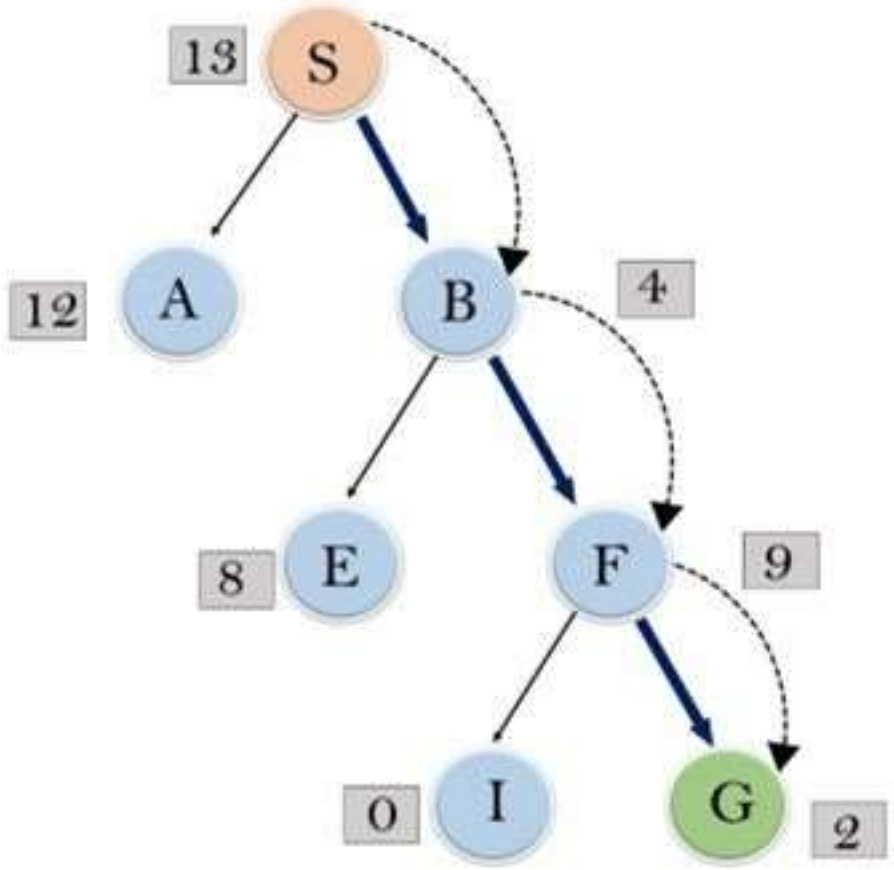
This algorithm is not optimal.





node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]

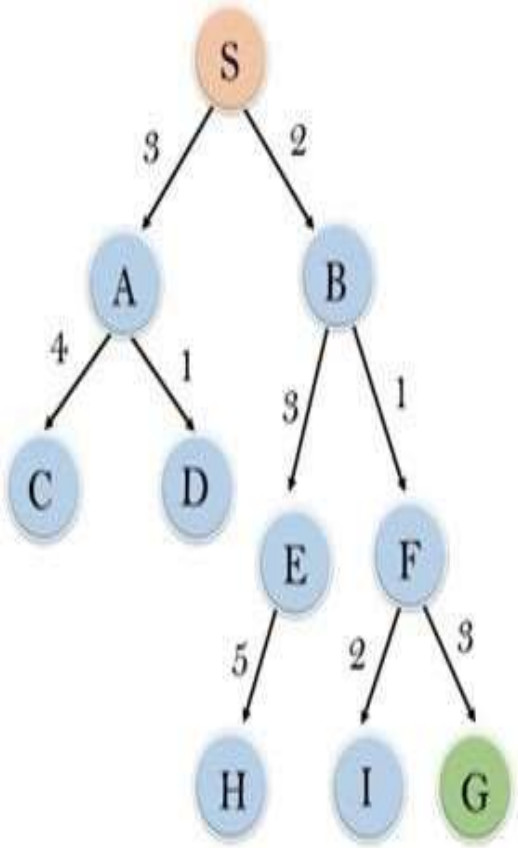
: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]

: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be:

S—> B—-->F—-> G



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

Hill Climbing

Hill climbing is a heuristic search algorithm that belongs to the family of local search methods. It is designed to solve problems where the goal is to find an optimal (or near-optimal) solution by iteratively moving from the current state to a better neighboring state, according to a heuristic or evaluation function.

Hill Climbing Algorithm

1. Evaluate the initial state.

If it is also the goal state, then return it; otherwise continue with the initial state as the current state.

2. Loop until the solution is found or until there are no new operators to be applied in the current state.

a) Select an operator that has not yet been applied to the current state and apply it to produce a new state.

b) Evaluate the new state.

i. If it is a goal state, then return it and quit.

ii. If it is not a goal state but it is better than the current state, then make it the current state.

iii. If it is not better than the current state, then continue in the loop.

Features of Hill Climbing Algorithm

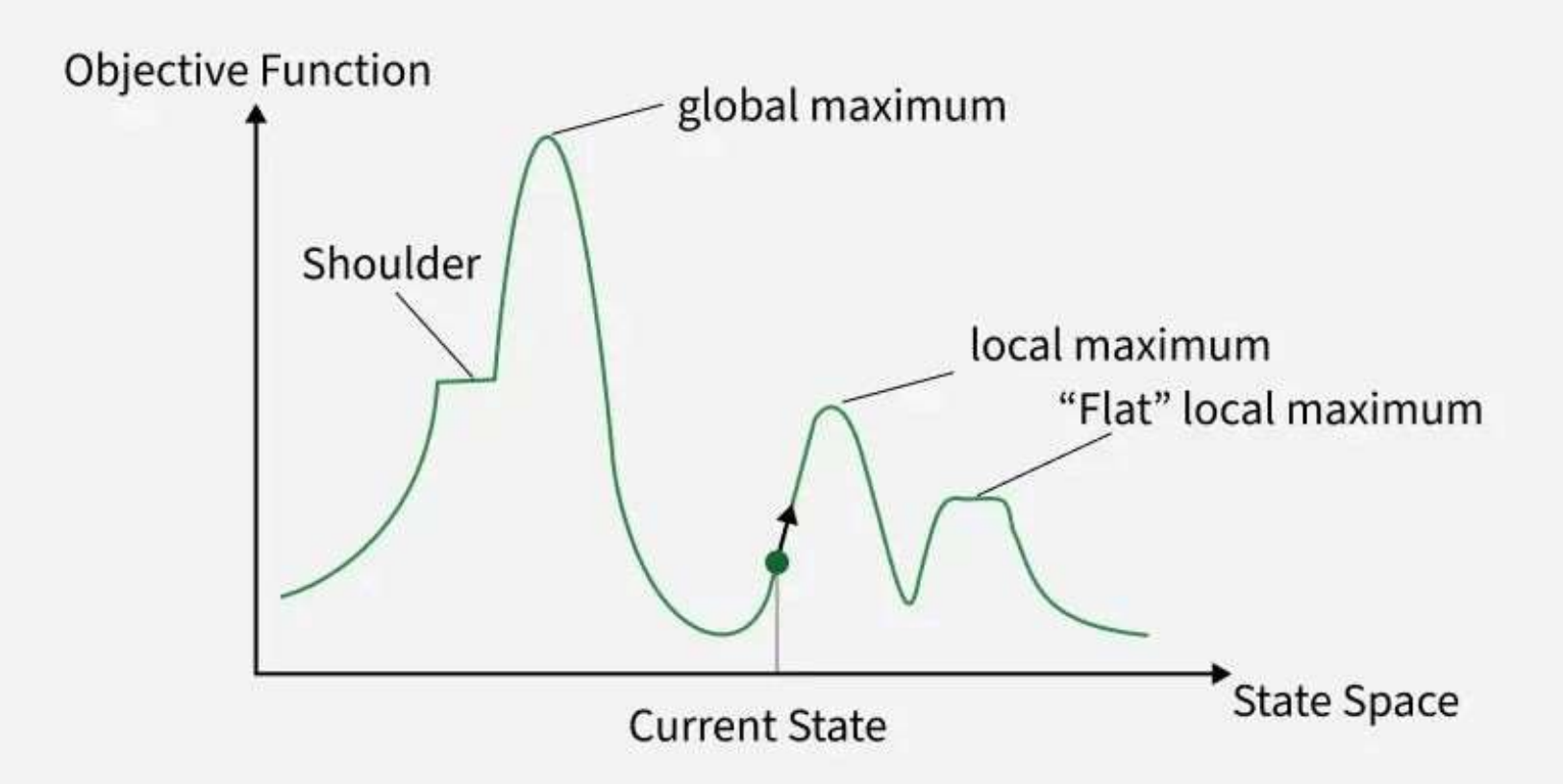
1. **Variant of Generating and Testing Algorithm**: Hill Climbing is a specific variant of the generating and testing algorithms. The process involves: This iterative feedback mechanism allows Hill Climbing to refine its search by using information from previous evaluations to inform future moves in the search space.

- **Generating possible solutions**: The algorithm creates potential solutions within the search space.
- **Testing solutions**: Each generated solution is evaluated to determine if it meets the desired criteria.
- **Iteration**: If a satisfactory solution is found, the algorithm terminates; otherwise, it returns to the generation step.

2. Greedy Approach: Hill Climbing algorithm uses greedy approach, meaning that at each step, it moves in the direction that optimizes the objective function. This strategy aims to find the optimal solution efficiently by making the best immediate choice without considering the overall problem context.

State-Space Diagram in Hill Climbing

The state-space diagram is a visual representation of all possible states the search algorithm can reach, plotted against the values of the objective function (the function we aim to maximize).


Objective Function
global maximum
Shoulder
local maximum
"Flat" local maximum
State Space
Current State

In the state-space diagram:

X-axis: Represents the state space which includes all the possible states or configurations that the algorithm can reach.
Y-axis: Represents the values of the objective function corresponding to each state.
The optimal solution in the state-space diagram is represented by the state where the **objective function** reaches its maximum value, also known as the **global maximum**.

Key Regions in the State-Space Diagram

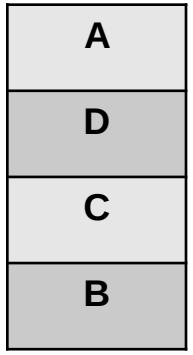
- **Local Maximum**: A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
- **Global Maximum**: The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
- **Plateau/Flat Local Maximum**: A plateau is a flat region where neighboring states have the same objective function value.

- **Ridge**: A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
- **Current State**: The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
- **Shoulder**: A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better solutions if it continues searching beyond the plateau difficult for the algorithm to decide on the best direction to move.

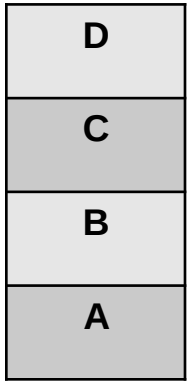
Hill Climbing Example

To understand the concept easily, we will take up a very simple example

Initial State



Goal State



Let us define such function h:

h(x) = +1 for all the blocks in the support structure if the block is correctly positioned; otherwise −1 for all the blocks in the support structure.

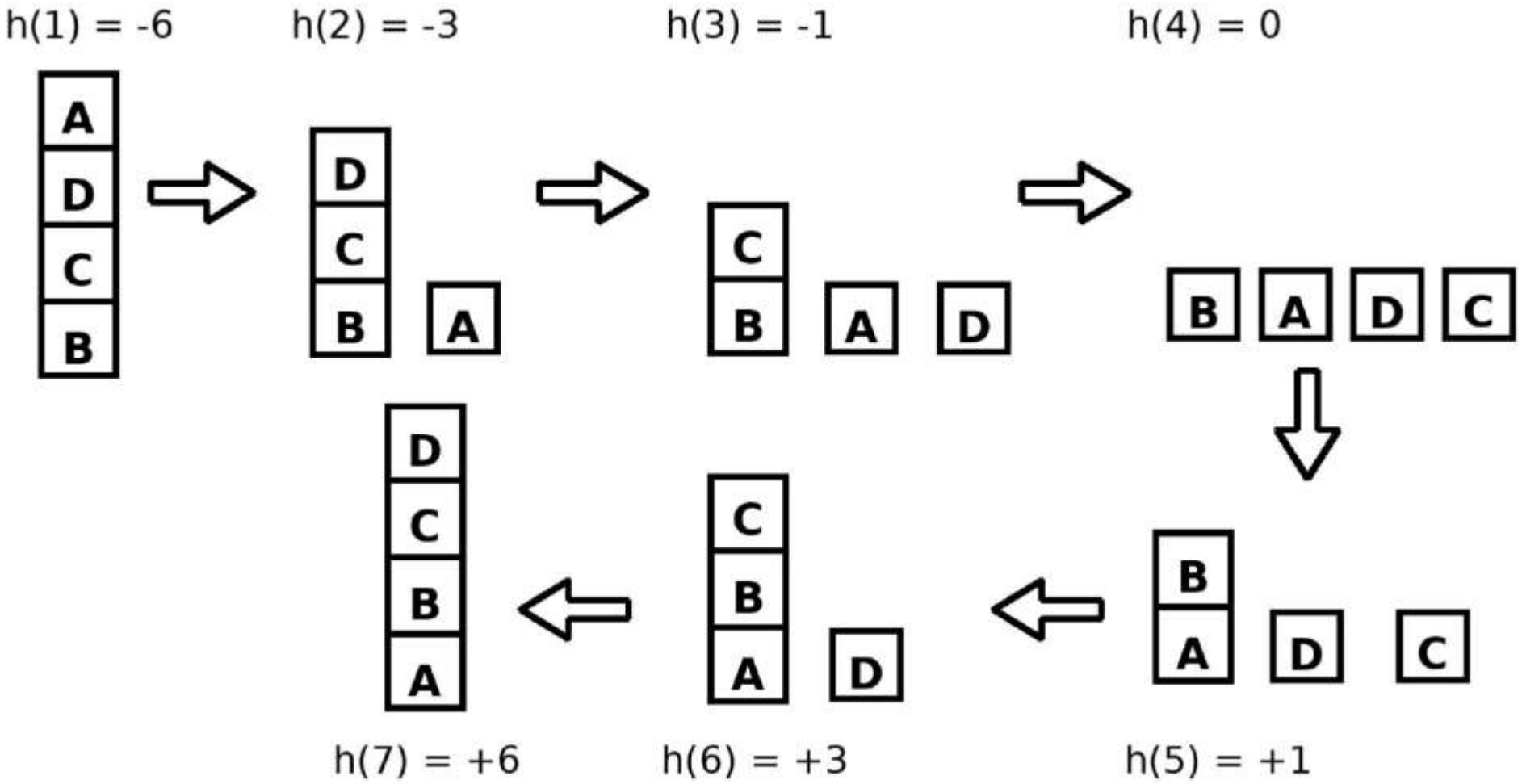
Heuristic Value Calculation

A	-3
D	-2
C	-1
B	0

Initial State **h=-6**

D	+3
C	+2
B	+1
A	0

Final State **h=+6**


h(1) = -6
h(2) = -3
h(3) = -1
h(4) = 0
A
D
C
B
D
C
B
A
C
B
A
D
B
A
D
C
B
A
D
C
D
C
B
A
C
B
A
D
B
A
D
C
h(7) = +6
h(6) = +3
h(5) = +1

A* Search

A* Search is an informed best-first search algorithm that efficiently determines the lowest cost path between any two nodes in a directed weighted graph with non-negative edge weights. This algorithm is a variant of Dijkstra’s algorithm.
A slight difference arises from the fact that an evaluation function is used to determine which node to explore next.

Evaluation Function

The evaluation function, **f(x)**, for the A* search algorithm is the following:

$$f(x) = g(x) + h(x)$$

Where **g(x)** represents the cost to get to node **x** and **h(x)** represents the estimated cost to arrive at the goal node from node **x**.

For the algorithm to generate the correct result, the evaluation function must be **admissible**, meaning that it never overestimates the cost to arrive at the goal node.

The Algorithm

The A* algorithm is implemented in a similar way to Dijkstra's algorithm. Given a weighted graph with non-negative edge weights, to find the lowest-cost path from a start node **S** to a goal node **G**, two lists are used:

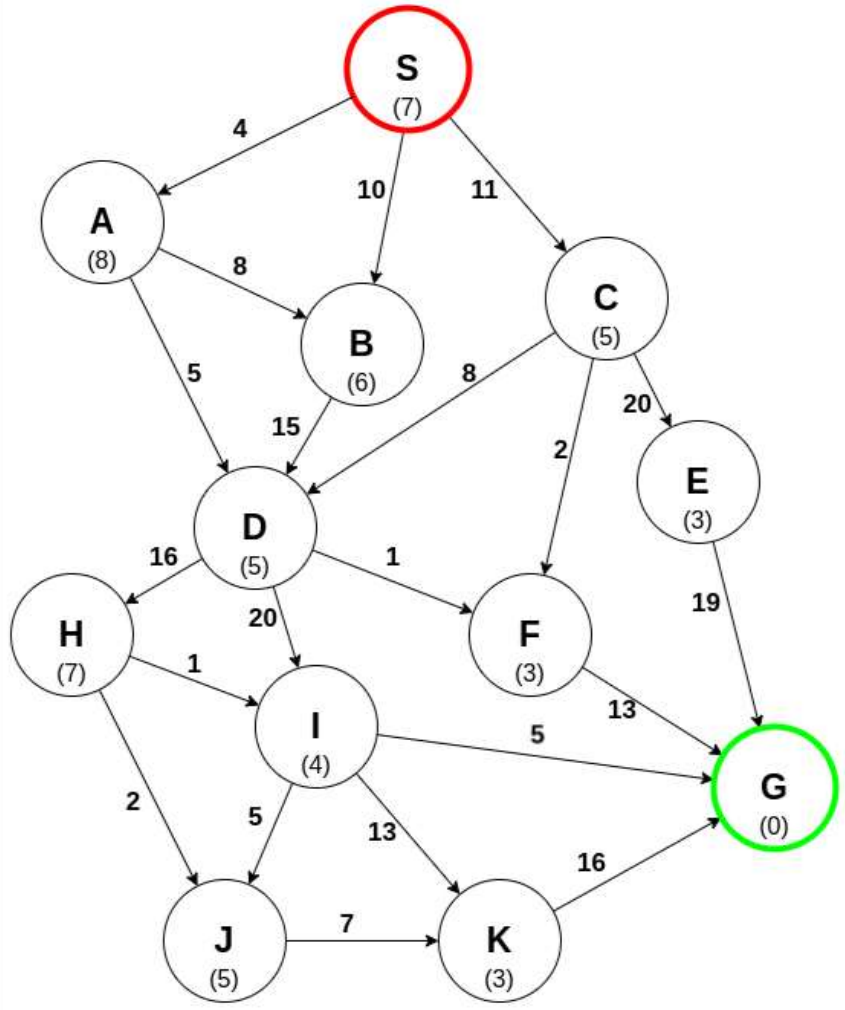
•An **open list**, implemented as a priority queue, which stores the next nodes to be explored. Because this is a priority queue, the most promising candidate node (the one with the lowest value from the evaluation function) is always at the top. Initially, the only node in this list is the start node **S**.

•A **closed list** which stores the nodes that have already been evaluated. When a node is in the closed list, it means that the lowest-cost path to that node has been found.

To find the lowest cost path, a search tree is constructed in the following way:

1.Initialize a tree with the root node being the start node **S**.
2.Remove the top node from the open list for exploration.
3.Add the current node to the closed list.
4.Add all nodes that have an incoming edge from the current node as child nodes in the tree.
5.Update the lowest cost to reach the child node.
6.Compute the evaluation function for every child node and add them to the open list.

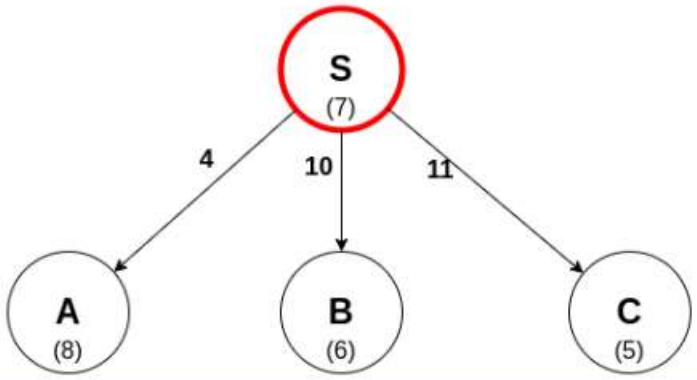
Example


S
(7)
A
(8)
B
(6)
C
(5)
D
(5)
E
(3)
F
(3)
G
(0)
H
(7)
I
(4)
J
(5)
K
(3)
4
10
11
8
5
15
8
20
2
16
1
20
19
1
13
5
2
5
13
16
7

Each edge has an associated weight, and each node has a heuristic cost (in parentheses).

An open list is maintained in which the node **S** is the only node in the list. The search tree can now be constructed.

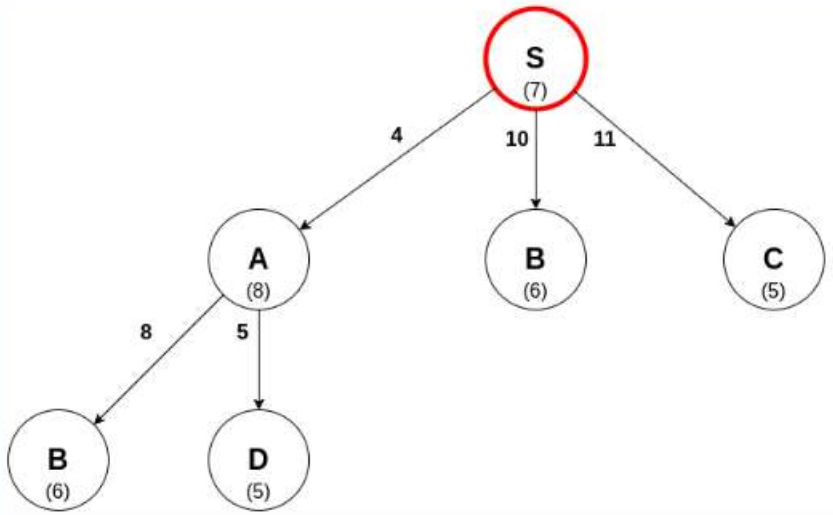
Exploring **S**:



Node[cost]
A[12]
B[16]
C[16]

Closed List
S

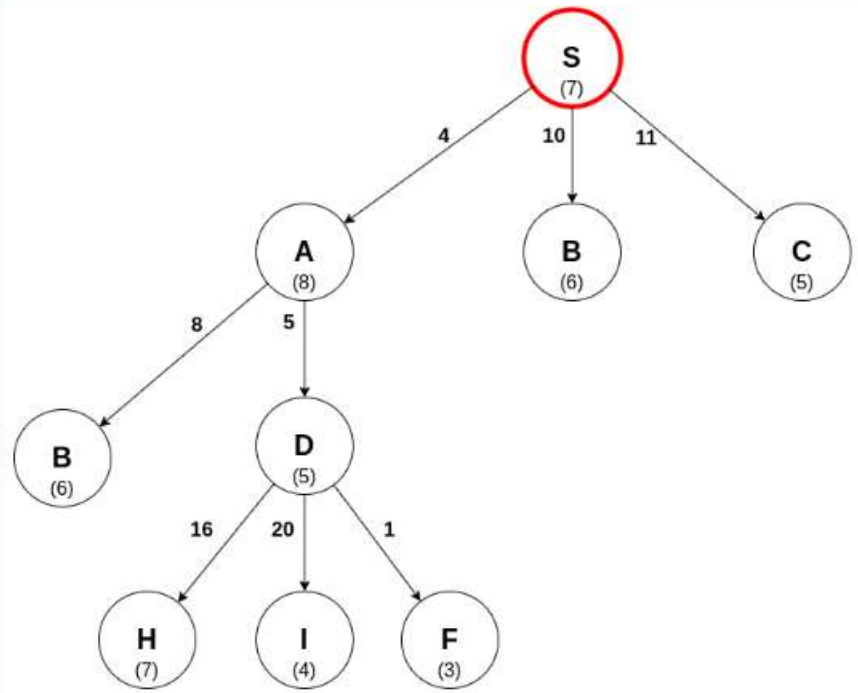
A is the current most promising path, so it is explored next:



Node[cost]
D[14]
C[16]
B[16]
B[18]

Closed List
S
A

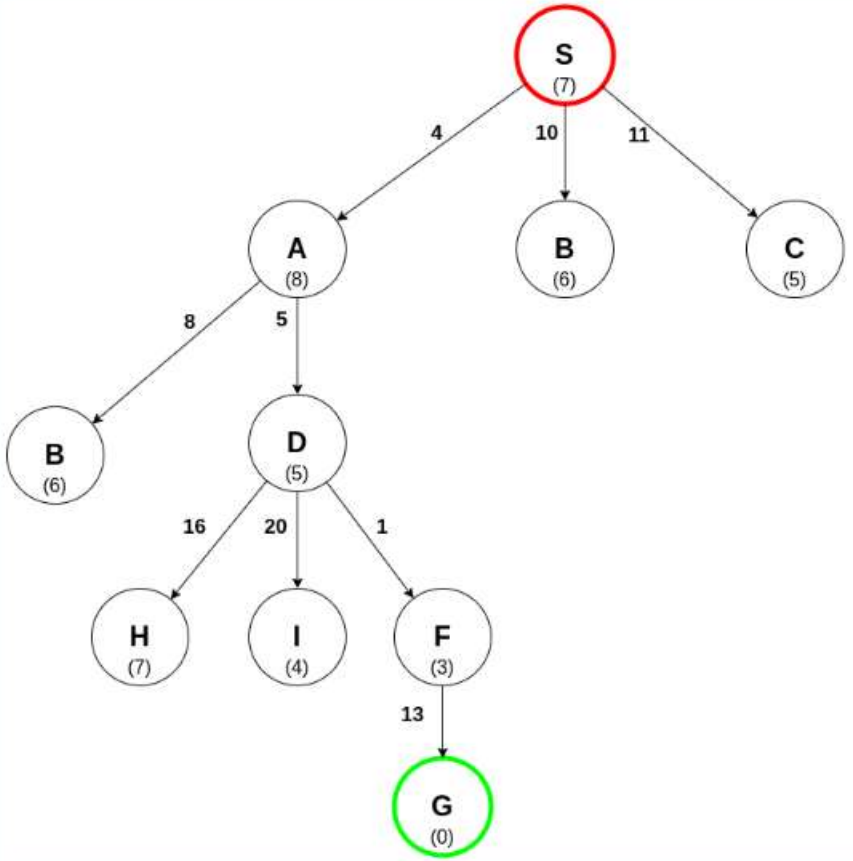
Exploring **D**:



Node[cost]
F[13]
C[16]
B[16]
H[32]
I[33]
B[18]

Closed List
S
A
D

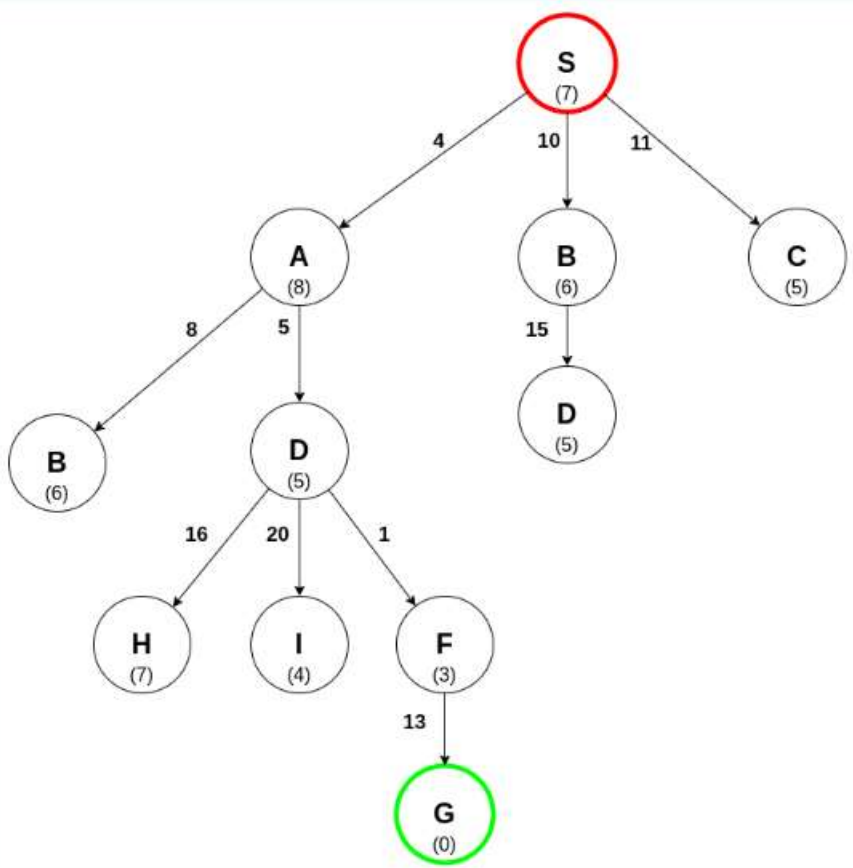
Exploring **F**:



Node[cost]
B[16]
C[16]
B[18]
H[32]
I[33]
G[23]

Closed List
S
A
D
F

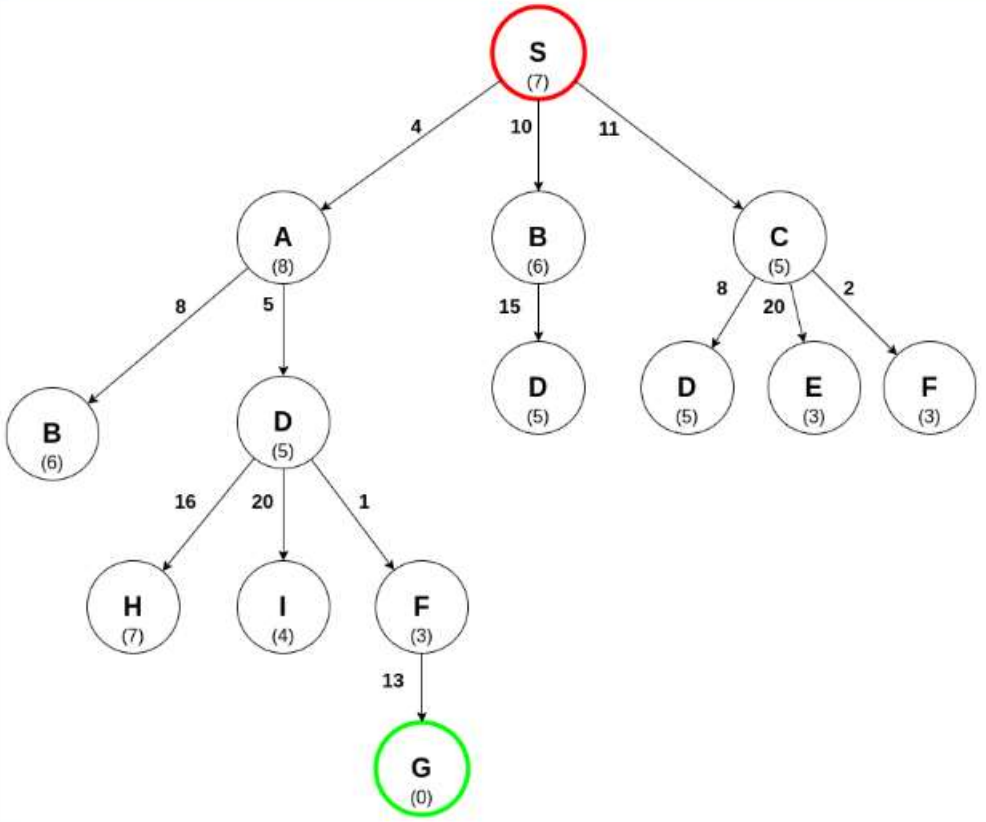
Notice that the goal node **G** has been found. However, it hasn't been explored, so the algorithm continues because there may be a shorter path to G. The node **B** has two entries in the open list: one at a cost of 16 (child of **S**) and one at a cost of 18 (child of **A**). The one with the lowest cost is explored next:



Node[cost]
C[16]
G[23]
B[18]
H[32]
I[33]

Closed List
S
A
D
F
B

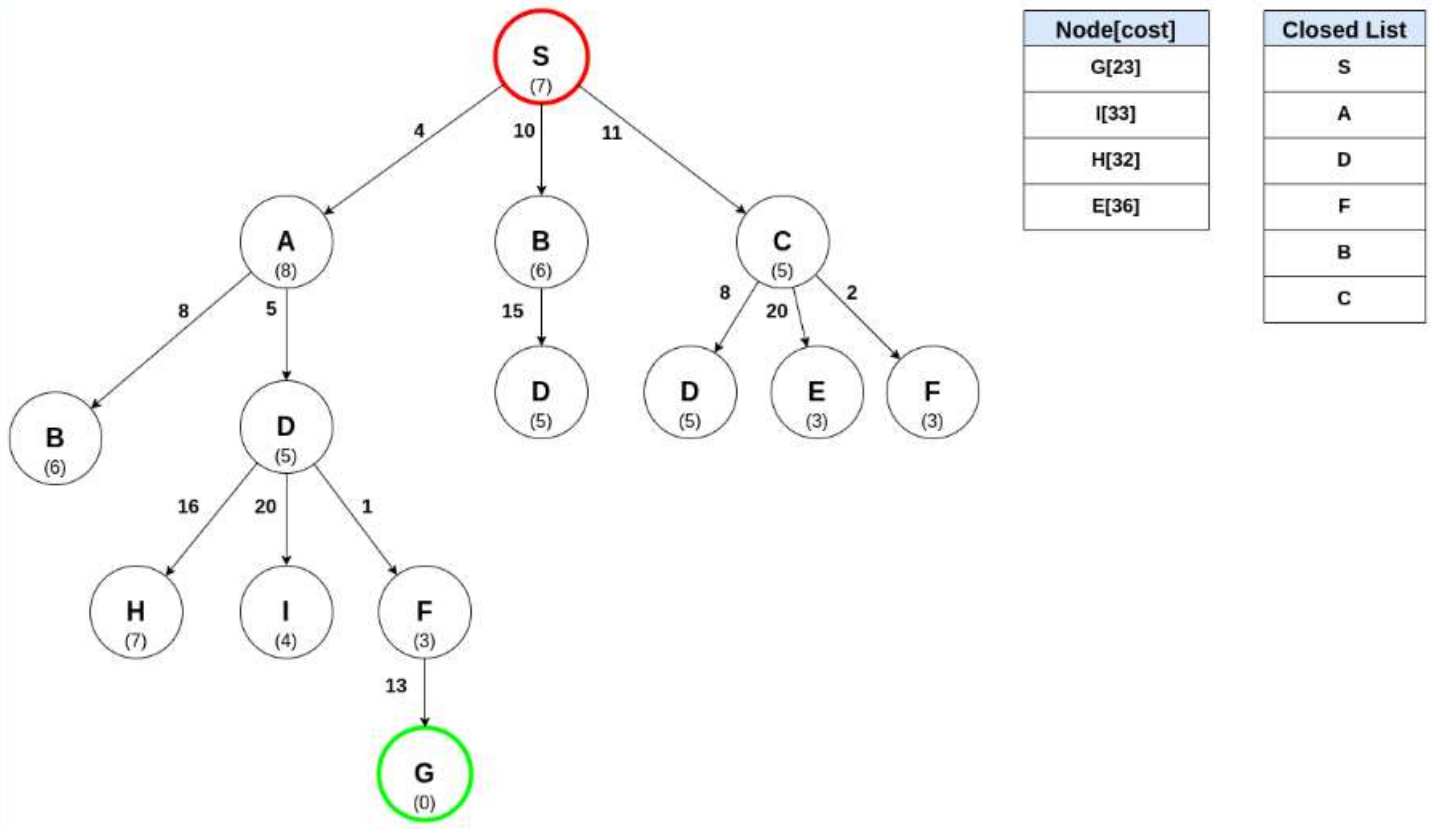
Exploring **C**:



Node[cost]
B[18]
G[23]
I[33]
H[32]
E[36]

Closed List
S
A
D
F
B
C

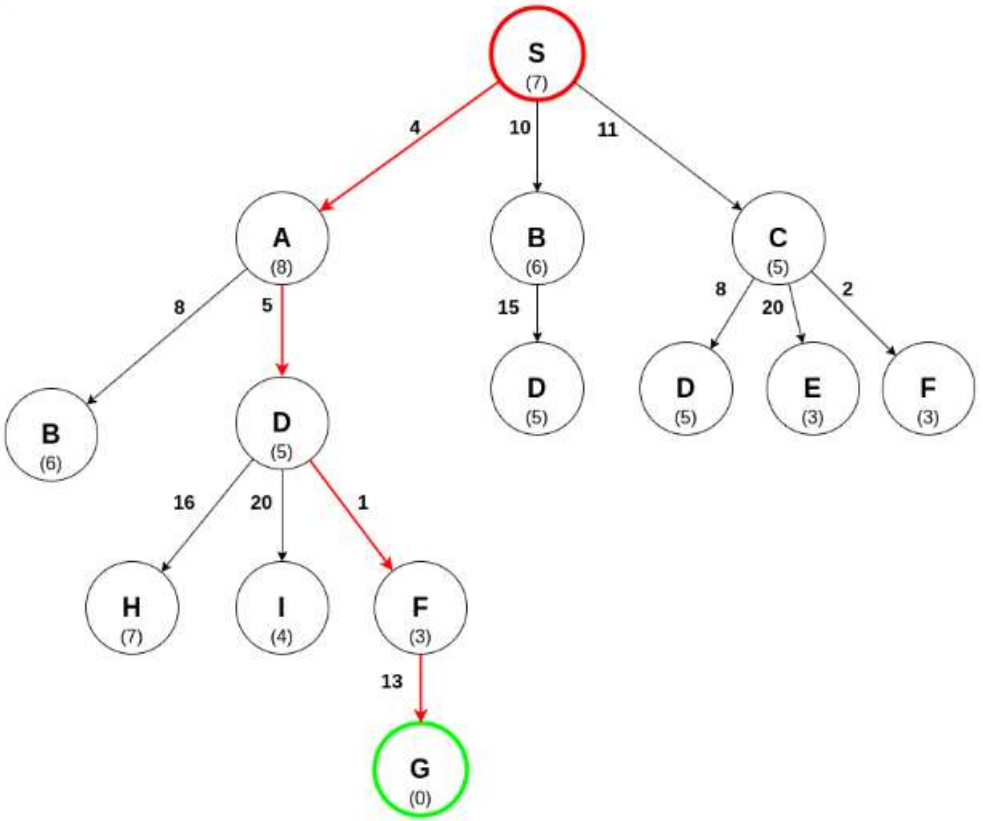
The next node in the open list is again **B**. However, because **B** has already been explored, meaning the shortest path to **B** has been found, it is not explored again, and the algorithm continues to the next candidate.



Node[cost]
G[23]
I[33]
H[32]
E[36]

Closed List
S
A
D
F
B
C

The next node to be explored is the goal node **G**, meaning the shortest path to **G** has been found! The path is constructed by tracing the graph backwards from **G** to **S**:



Node[cost]
I[33]
H[32]
E[36]

Closed List
S
A
D
F
B
C
G

Feature	Best-First Search (Greedy)	A* Search
Evaluation function	`f(n) = h(n)`	`f(n) = g(n) + h(n)`
Considers path cost `g(n)`	No	Yes
Considers heuristic `h(n)`	Yes	Yes
Main idea	Choose the node that appears closest to the goal	Choose the node with the lowest total estimated cost
Optimal solution	Not guaranteed	Guaranteed if `h(n)` is admissible/consistent
Speed	Often faster	May explore more nodes
Memory	Can be high	Can be high
Example	Select node with smallest estimated distance to destination	Select node with smallest `cost so far + estimated remaining cost`

Constraint Satisfaction Problem (CSP)

- A **Constraint Satisfaction Problem (CSP)** is a mathematical problem defined by a set of objects whose state must satisfy a number of constraints or limitations. Unlike standard search problems where the state is a "black box," a CSP state is defined by a set of variables, and the goal is found when each variable has a value that satisfies all constraints.

Core Components of a CSP

- A CSP consists of three main components:
- **Variables (X):** A set of variables, {X_1, X_2, ……., X_n}.
- **Domains (D):** A set of domains, {D_1, D_2, ……..D_n}, where each variable X_i has a corresponding domain D_i of possible values it can take.
- **Constraints (C):** A set of constraints, {C_1, C_2,……., C_m}, that specify allowable combinations of values for subsets of variables. Each constraint involves a subset of variables and a relation that defines which value combinations are valid.

Solving CSPs

A CSP is solved when every variable is assigned a value from its domain (**complete assignment**) in a way that satisfies every constraint (**consistent assignment**).

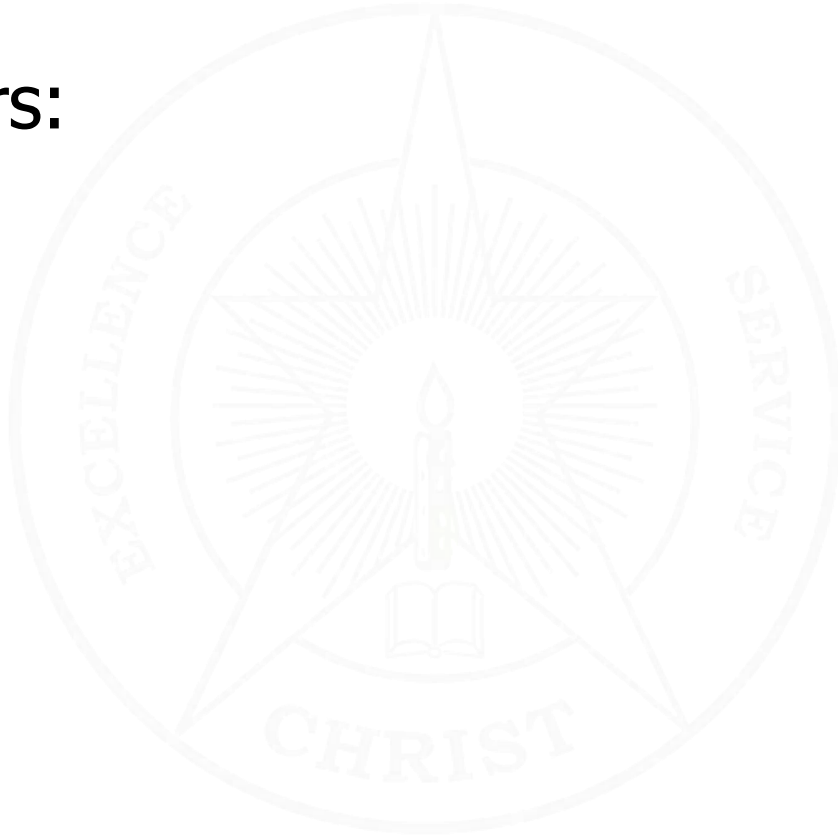
- **State Representation:** A state is defined by an assignment of values to some or all variables.
- **Goal Test:** The assignment is complete and satisfies all constraints.
- **Conflict Handling:** If a value assignment leads to a "conflict" (where a constraint is violated), the agent must backtrack or try different value combinations until a valid solution is found.
- **Complexity:** The efficiency of solving a CSP can be evaluated by the **number of symbols** (variables) and the **number of clauses** (constraints). As the number of variables and constraints increases, the search space grows exponentially.

Example: Map Coloring Problem

Suppose we have three regions: **A, B, and C**.

Each region must be colored using one of these colors:
Red, Green, Blue

But the rule is:
Adjacent regions should not have the same color.



Variables

The variables are:

A, B, C

Domains

Each variable can take any of the following values:

A ={Red, Green, Blue}

B ={Red, Green, Blue}

C ={Red, Green, Blue}

Constraints

Assume:

A is adjacent to B
B is adjacent to C
A is adjacent to C

So the constraints are:

A ≠ B
B ≠ C
A ≠ C

One Possible Solution

A = Red
B = Green
C = Blue

This is a valid solution because no two adjacent regions have the same color.

Real-Life Examples of CSP

APPLICATION	CONSTRAINT
Timetable scheduling	Two classes cannot use the same room at the same time
Sudoku	Numbers 1 to 9 should not repeat in a row, column, or box
Map coloring	Neighboring regions should not have the same color
Job scheduling	A task must be completed before another task starts
Exam scheduling	A student should not have two exams at the same time

Means-Ends Analysis in Artificial Intelligence

- We have studied the strategies which can reason either in forward or backward, but a mixture of the two directions is appropriate for solving a complex and large problem. Such a mixed strategy, make it possible that first to solve the major part of a problem and then go back and solve the small problems arise during combining the big parts of the problem. Such a technique is called **Means-Ends Analysis**.
- Means-Ends Analysis is problem-solving techniques used in Artificial intelligence for limiting search in AI programs.
- It is a mixture of Backward and forward search technique.
- The MEA technique was first introduced in 1961 by Allen Newell, and Herbert A. Simon in their problem-solving computer program, which was named as General Problem Solver (GPS).
- The MEA analysis process centered on the evaluation of the difference between the current state and goal state.

How means-ends analysis Works:

The means-ends analysis process can be applied recursively for a problem. It is a strategy to control search in problem-solving. Following are the main Steps which describes the working of MEA technique for solving a problem.

- First, evaluate the difference between Initial State and final State.
- Select the various operators which can be applied for each difference.
- Apply the operator at each difference, which reduces the difference between the current state and goal state.

Operator Subgoaling

In the MEA process, we detect the differences between the current state and goal state. Once these differences occur, then we can apply an operator to reduce the differences. But sometimes it is possible that an operator cannot be applied to the current state. So we create the subproblem of the current state, in which operator can be applied, such type of backward chaining in which operators are selected, and then sub goals are set up to establish the preconditions of the operator is called **Operator Subgoaling**.

Algorithm for Means-Ends Analysis:

Let's we take Current state as CURRENT and Goal State as GOAL, then following are the steps for the MEA algorithm.

Step 1: Compare CURRENT to GOAL, if there are no differences between both then return Success and Exit.

Step 2: Else, select the most significant difference and reduce it by doing the following steps until the success or failure occurs.

Select a new operator O which is applicable for the current difference, and if there is no such operator, then signal failure.

Attempt to apply operator O to CURRENT. Make a description of two states.
i) O-Start, a state in which O's preconditions are satisfied.
ii) O-Result, the state that would result if O were applied In O-start.

If
(First-Part ←— MEA (CURRENT, O-START)
And
(LAST-Part ←— MEA (O-Result, GOAL), are successful, then signal Success and return the result of combining FIRST-PART, O, and LAST-PART

Step 1: Compare CURRENT to GOAL

CURRENT represents the present situation.

GOAL represents the situation we want to achieve.

The algorithm checks what differences exist between them.

Example:

CURRENT: You are at home.

GOAL: You are at college.

Difference: Your location is not the college.

- **"If there are no differences between both, return Success and Exit"**
- If the CURRENT state is already the same as the GOAL state, the problem has been solved.
- Therefore:
- Return **Success**.
- Stop the algorithm.
- **Example:**
- CURRENT: At college.
- GOAL: At college.
- There is no difference, so the algorithm ends successfully.

Step 2: Select the most significant difference

When several differences exist, choose the most important difference first.

Example:

- Suppose your goal is to attend a class at college, but:
- You are at home.
- You are not dressed.
- You do not have your bag.
- The most significant difference may be that you are still at home. However, before travelling, you may first need to get ready and pack your bag.

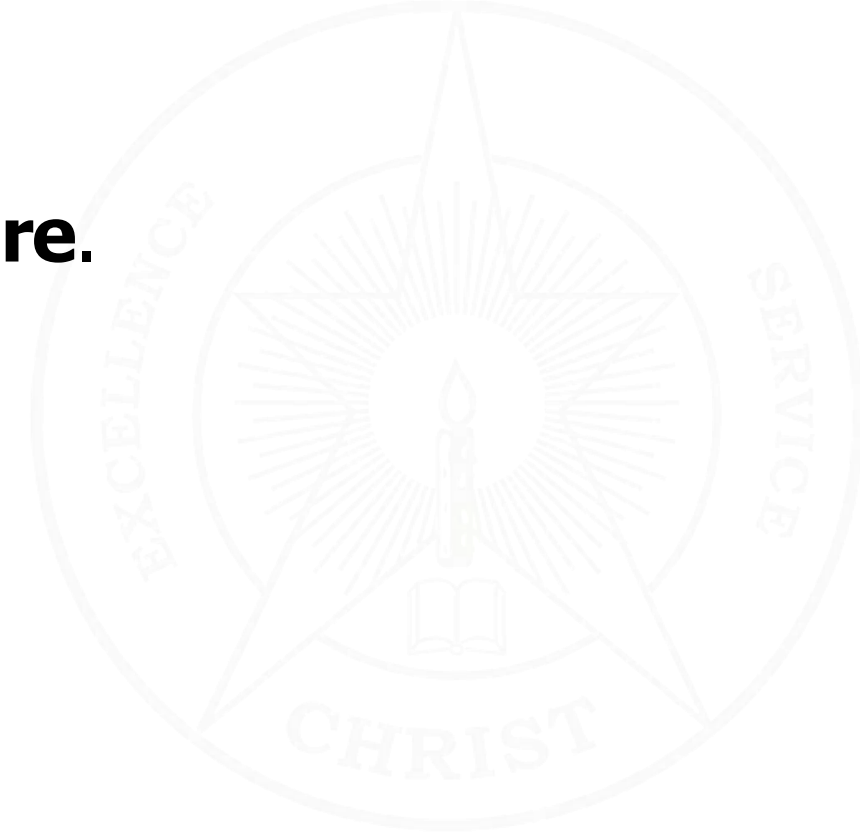
Reduce it by doing the following steps until success or failure occurs

The algorithm tries to reduce or remove the selected difference by choosing an appropriate action.

It continues until:

The goal is reached, producing **Success**, or

No suitable action can be found, producing **Failure**.

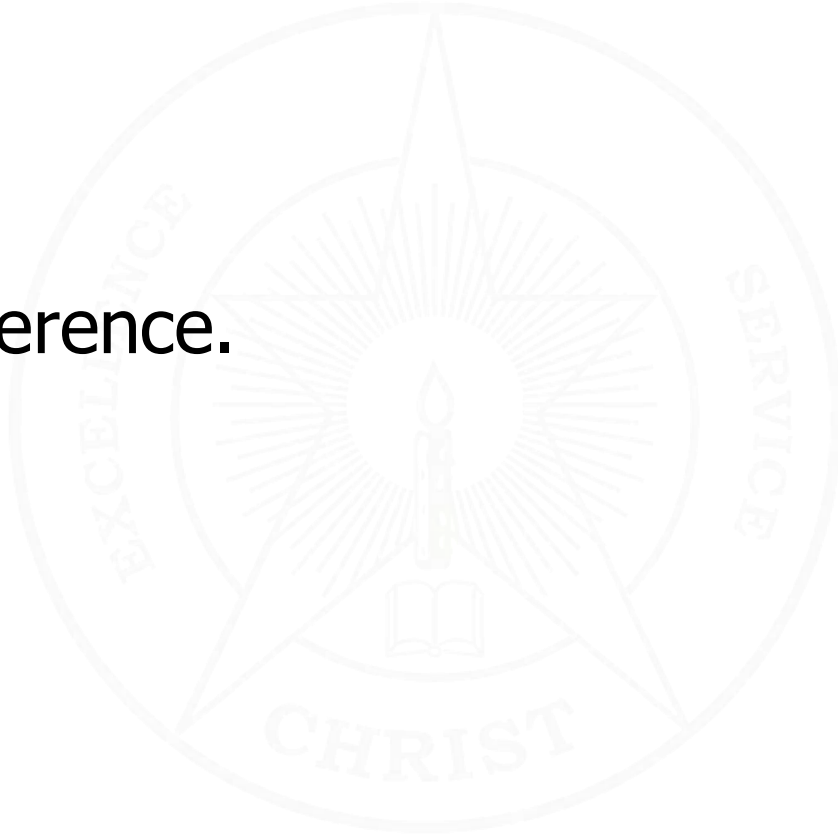


Select a new operator O which is applicable for the current difference

An **operator** is an action that changes one state into another state.
Here, **O** represents the selected operator.

Example:

Difference: You are at home instead of college.
Operator O: Travel by bus to college.
The operator should help reduce the selected difference.



If there is no such operator, signal Failure

Sometimes no available action can solve the current difference.
In that situation, the algorithm cannot continue and returns **Failure**.

Example:

CURRENT: You are at home.
GOAL: You must reach another city immediately.
No transport is available.
Because no suitable operator exists, the algorithm signals failure.

Attempt to apply operator O to CURRENT

The algorithm now tries to perform the selected action on the current state. However, an operator may require certain conditions to be satisfied before it can be applied.

Example:

To use the operator:

Travel by bus

You may need:

Money for the ticket.

A bus available.

Access to the bus stop.

Make a description of two states

Before applying the operator, the algorithm identifies two important states:

O-Start

O-Result

O-Start: A state in which O's preconditions are satisfied

O-Start is the state in which all requirements for applying operator O are satisfied.

A **precondition** is something that must already be true before an action can be performed.

Example:

Operator O: Travel by bus.

Preconditions:

You are at the bus stop.

You have money or a travel card.

A bus is available.

Therefore, O-Start is:

You are at the bus stop with a valid ticket or travel card, and the bus is available.

O-Result: The state that would result if O were applied in O-Start

O-Result is the new state produced after applying operator O.

Example:

O-Start: You are at the bus stop with a ticket.

Operator O: Take the bus to college.

O-Result: You are at college.

FIRST-PART ← MEA(CURRENT, O-START)

This line calls the MEA algorithm again.

It tries to change the CURRENT state into the O-Start state.

This is necessary because the operator cannot be used until all its preconditions are satisfied.

Example:

CURRENT: You are at home.

O-Start: You must be at the bus stop with a ticket.

MEA must first find actions such as:

1. Take your wallet.
2. Walk to the bus stop.
3. Purchase a ticket.

These actions form the **FIRST-PART** of the solution.

And LAST-PART ← MEA(O-Result, GOAL)

After operator O has been applied, the algorithm checks whether the resulting state reaches the final goal.

It calls MEA again to move from O-Result to GOAL.

Example:

O-Result: You have reached the college gate.

GOAL: You are sitting in the classroom.

The algorithm may select additional actions:

1. Enter the college building.
2. Walk to the classroom.
3. Sit in the class.

These actions form the **LAST-PART** of the solution.

If FIRST-PART and LAST-PART are successful

The complete solution is successful only when both parts succeed:

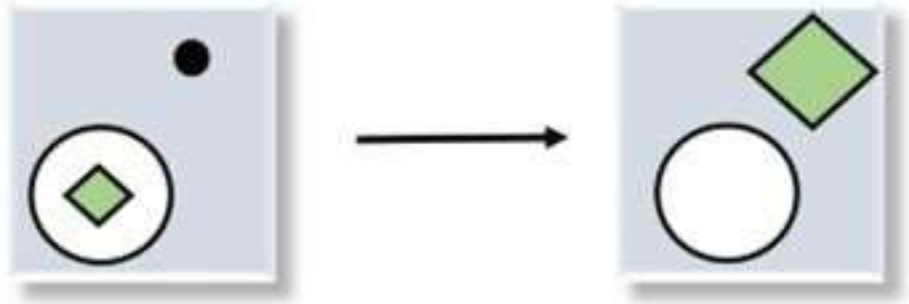
- **FIRST-PART:** Successfully changes CURRENT into O-Start.
- **LAST-PART:** Successfully changes O-Result into GOAL.

If either part fails, the selected operator cannot produce a complete solution.

- **"Signal Success"**
- When both recursive MEA calls succeed, the algorithm reports that it has found a valid solution.

Example of Mean-Ends Analysis:

Let's take an example where we know the initial state and goal state as given below. In this problem, we need to get the goal state by finding differences between the initial state and goal state and applying operators.



Solution:

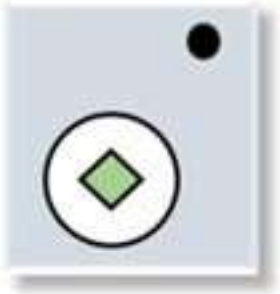
To solve the above problem, we will first find the differences between initial states and goal states, and for each difference, we will generate a new state and will apply the operators. The operators we have for this problem are:

Move

Delete

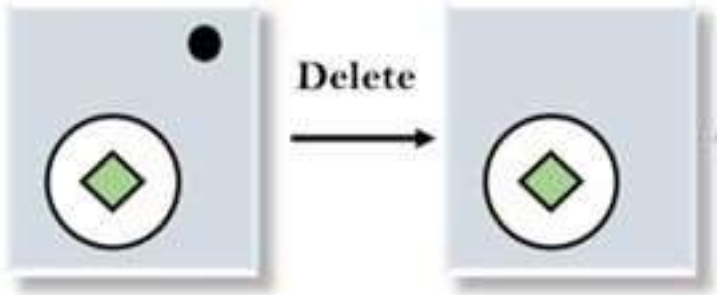
Expand

1. Evaluating the initial state: In the first step, we will evaluate the initial state and will compare the initial and Goal state to find the differences between both states.



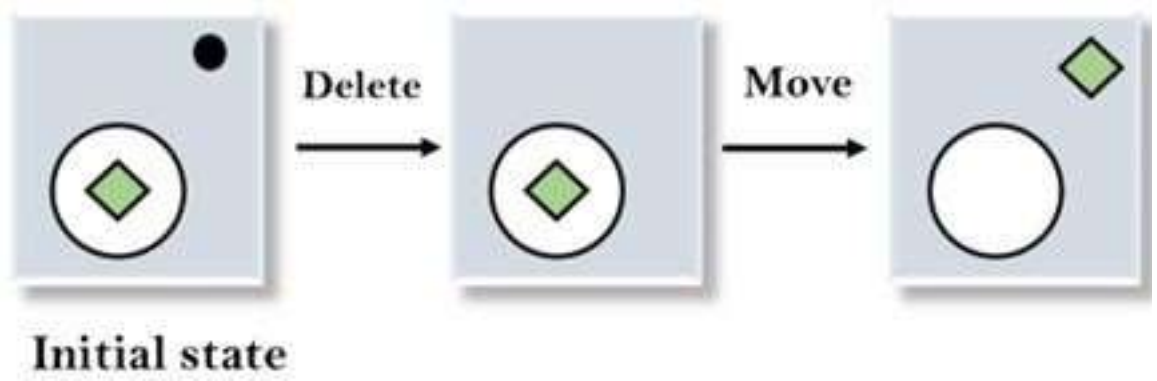
Initial state

2. Applying Delete operator: As we can check the first difference is that in goal state there is no dot symbol which is present in the initial state, so, first we will apply the **Delete operator** to remove this dot.



Initial state

3. Applying Move Operator: After applying the Delete operator, the new state occurs which we will again compare with goal state. After comparing these states, there is another difference that is the square is outside the circle, so, we will apply the **Move Operator**.



4. Applying Expand Operator: Now a new state is generated in the third step, and we will compare this state with the goal state. After comparing the states there is still one difference which is the size of the square, so, we will apply **Expand operator**, and finally, it will generate the goal state.

